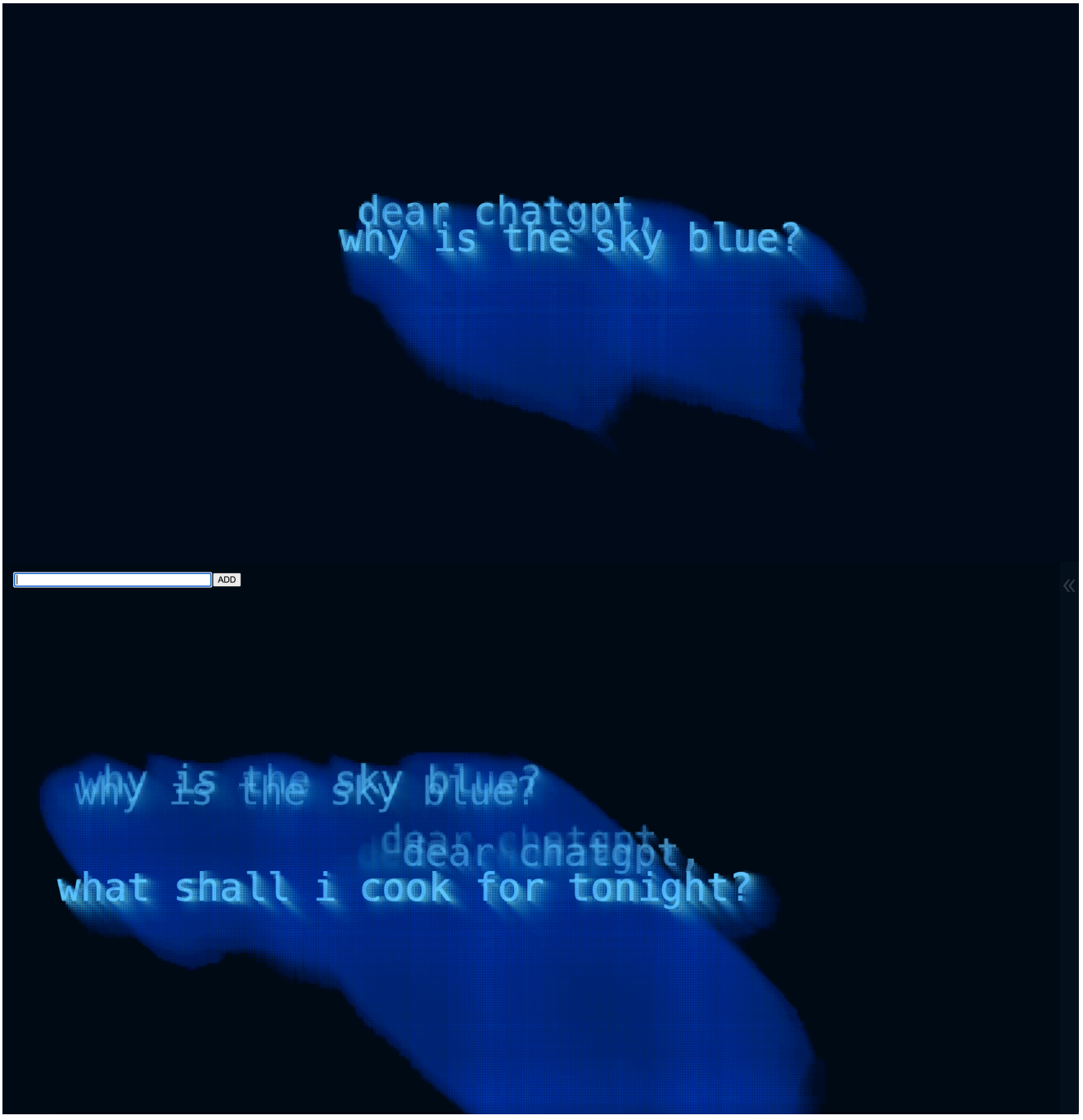
week-6

Day 1

Final Presentation



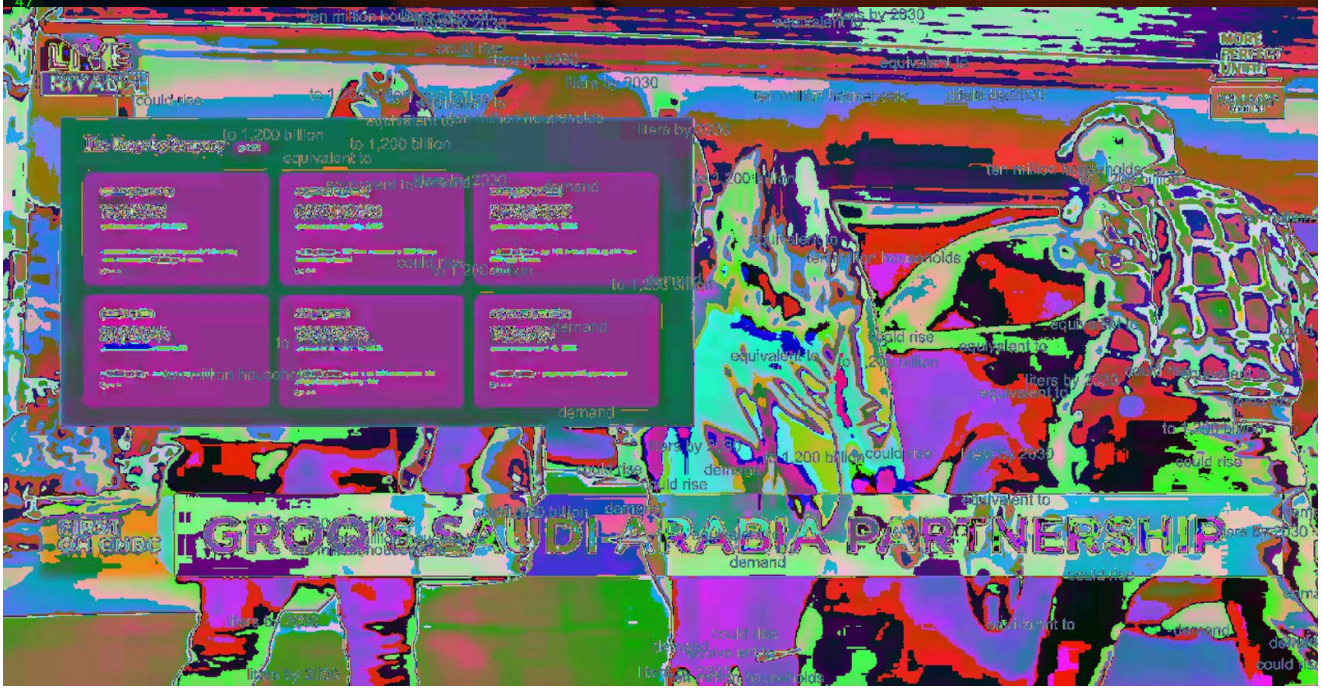
```
      t.x += random(-10, 50);
      t.y += random(-20, 20);
    }

    let halfWidth = t.calculatedWidth / 2;
    t.x = constrain(t.x, halfWidth + 20, width - (halfWidth + 20));
    t.y = constrain(t.y, 100, height - 100);

    textSize(t.size);
    text(t.word, t.x, t.y);
  }

  loadPixels();
  for (let y = 0; y < height; y += 4) {
    for (let x = 0; x < width; x += 4) {
      let xOffset = sin(frameCount * 0.05 + y * 0.02) * 4;
      let yOffset = cos(frameCount * 0.05 + x * 0.02) * 4;

      let srcX = constrain(floor(x + xOffset), 0, width - 1);
      let srcY = constrain(floor(y + yOffset), 0, height - 1);

      let pixelColor = get(srcX, srcY);

      if (brightness(pixelColor) > 20) {
        fill(
          red(pixelColor)+30,
          green(pixelColor) +10,
          blue(pixelColor) +40,
          35
        );
        noStroke();
        rect(x, y, 5, 5);
```

ADD

can i put lime instead of lemon in this recipe?
do my friends hate me?
what is a paradox?
what do we use water for?
is there enough water?
what shall i cook for dinner?


```
      t.x += random(-10, 50);
      t.y += random(-20, 20);
    }

    let halfWidth = t.calculatedWidth / 2;
    t.x = constrain(t.x, halfWidth + 20, width - (halfWidth + 20));
    t.y = constrain(t.y, 100, height - 100);

    textSize(t.size);
    text(t.word, t.x, t.y);
  }

  loadPixels();
  for (let y = 0; y < height; y += 4) {
    for (let x = 0; x < width; x += 4) {
      let xOffset = sin(frameCount * 0.05 + y * 0.02) * 4;
      let yOffset = cos(frameCount * 0.05 + x * 0.02) * 4;

      let srcX = constrain(floor(x + xOffset), 0, width - 1);
      let srcY = constrain(floor(y + yOffset), 0, height - 1);

      let pixelColor = get(srcX, srcY);

      if (brightness(pixelColor) > 20) {
        fill(
          red(pixelColor)+30,
          green(pixelColor) +10,
          blue(pixelColor) *0.1,
          35
        );
        noStroke();
        rect(x, y, 5, 5);
```

```
  textFont('monospace');
  textAlign(CENTER, CENTER);

  background(5, 10, 25);

  input = createInput();
  input.position(20, 20);
  input.size(300);

  button = createButton('ADD');
  button.position(330, 20);

  button.mousePressed(function(event) {
    addNewText();
    event.stopPropagation();
  });

  input.changed(addNewText);

  updateLiveAudio();
}

function draw() {
  background(5, 10, 25, 20);

  for (let t of texts) {
    t.alpha *= 0.992;
    fill(13, t.alpha);

    t.x += noise(frameCount * 0.01 + t.x) * 2 - 1;
    t.y += noise(frameCount * 0.01 + t.y) * 2 - 1;
```



+CO Creative Coding

AI’s Hidden Water Footprint

According to the Swissinfo article «How AI data centers risk straining Switzerland’s water resources» by Sara Ibrahim, the official data on water consumption remains scarce. However, the International Energy Agency estimates current demand at around 560 billion liters per year, which could escalate to 1,200 billion liters by 2030—a volume equivalent to the annual water needs of roughly ten million households.

These numbers hit a sore spot, especially when thinking about the impact that all of this could have on us. Therefore, the topic seems very relevant and we wanted to raise awareness about it through our project.

We did a lot of research and were flooded with news articles but as already mentioned at the beginning, not much of the data is open to the public. This brought up even more questions and left us with uncertainty not only about the topic but about the future as well.

So we started where most of us started at the rise of AI: ChatGPT. For most of us it is a helpful tool to solve our “problems”. A lot of us use it daily and most of the time we don’t think about the impact of a single question. So I’ve made it my mission to write down random questions and incorporate them into my code. As time goes on, the questions become more and more serious and the water slowly turns brown, in order to draw attention to the water pollution caused by data center cooling. To reinforce this shift, I’ve added a fitting audio track that emphasizes the gravity of the issue and creates a specific, tense atmosphere. Ultimately, this leads to the central question: Is there actually enough water?

```
// {"P5LIVE":{"name":"01_water","mod":1780350229461}}

let texts = [];
let input;
let button;

let extAudioCtx;
let extAudioElement;
```

```
let extAnalyser;
let extSource;
let extDataArray;
let extAudioStarted = false;
let extGainNode;

function setup() {
  createCanvas(windowWidth, windowHeight);

  textFont('monospace');
  textAlign(CENTER, CENTER);

  background(5, 10, 25);

  // setup elements (input + button)
  input = createInput();
  input.position(20, 20);
  input.size(300);

  button = createButton('ADD');
  button.position(330, 20);

  // click event for the button
  button.mousePressed(function(event) {
    addNewText();
    event.stopPropagation(); // stops click from triggering canvas events
below
  });

  input.changed(addNewText); // triggers on enter key inside input

  updateLiveAudio(); // start audio loop
}

function draw() {
  // slight alpha background creates the water effect
  background(5, 10, 25, 20);

  // update and draw all text objects
  for (let t of texts) {
    t.alpha *= 0.992; // slow fade out
    fill(100,200,255, t.alpha);

    // organic movement using perlin noise (organic texture)
    t.x += noise(frameCount * 0.01 + t.x) * 2 - 1;
    t.y += noise(frameCount * 0.01 + t.y) * 2 - 1;

    // random tiny skips
    if (random() < 0.02) {
      t.x += random(-10, 50);
```

```
      t.y += random(-20, 20);
    }

    // keep text within screen bounds
    let halfWidth = t.calculatedWidth / 2;
    t.x = constrain(t.x, halfWidth + 20, width - (halfWidth + 20));
    t.y = constrain(t.y, 100, height - 100);

    textSize(t.size);
    text(t.word, t.x, t.y);
  }

  // pixel effect / wavy distortion
  loadPixels();
  for (let y = 0; y < height; y += 4) {
    for (let x = 0; x < width; x += 4) {
      // sin/cos for the water movement look
      let xOffset = sin(frameCount * 0.05 + y * 0.02) * 4;
      let yOffset = cos(frameCount * 0.05 + x * 0.02) * 4;

      let srcX = constrain(floor(x + xOffset), 0, width - 1);
      let srcY = constrain(floor(y + yOffset), 0, height - 1);

      let pixelColor = get(srcX, srcY);

      // if pixel is bright enough, draw a colored rect over it (water
effect)
      if (brightness(pixelColor) > 20) {
        fill(
          red(pixelColor),
          green(pixelColor) +15,
          blue(pixelColor) +40,
          35
        );
        noStroke();
        rect(x, y, 5, 5);
      }
    }
  }

  // clean up invisible text
  texts = texts.filter(t => t.alpha > 3);
}

function addNewText() {
  let newWord = input.value();

  if (newWord.trim() !== '') {
    let currentSize = 60;
    textSize(currentSize);
```

```
    let measuredWidth = textWidth(newWord);
    let maxWidth = width - 100;

    // scale down text if it's too wide for the screen
    if (measuredWidth > maxWidth) {
      currentSize = currentSize * (maxWidth / measuredWidth);
      textSize(currentSize);
      measuredWidth = textWidth(newWord);
    }

    let halfWidth = measuredWidth / 2;
    let minX = halfWidth + 50;
    let maxX = width - (halfWidth + 50);

    // push new word object into array
    texts.push({
      word: newWord,
      x: random(minX, maxX),
      y: random(150, height - 150),
      alpha: 255,
      size: currentSize,
      calculatedWidth: measuredWidth
    });

    input.value('');

    if (!extAudioStarted) {
      startAudioSystem();
    }
  }
}

function windowResized() {
  resizeCanvas(windowWidth, windowHeight);
}

function mousePressed() {
  // ignore if clicking top-left button area
  if (mouseX < 380 && mouseY < 60) {
    return;
  }
  if (!extAudioStarted) {
    startAudioSystem();
  }
}

// Web Audio setup for mp3 file
function startAudioSystem() {
  extAudioCtx = new (window.AudioContext || window.webkitAudioContext)();
```

```
  extAudioElement = document.createElement('audio');
  extAudioElement.src = 'data/water2.mp3';
  extAudioElement.loop = true;
  extAudioElement.crossOrigin = "anonymous";

  extSource = extAudioCtx.createMediaElementSource(extAudioElement);
  extAnalyser = extAudioCtx.createAnalyser();
  extAnalyser.fftSize = 256;
  let bufferLength = extAnalyser.frequencyBinCount;
  extDataArray = new Uint8Array(bufferLength);

  extGainNode = extAudioCtx.createGain();

  // routing: Source -> Gain (Volume) -> Analyser -> Output
  extSource.connect(extGainNode);
  extGainNode.connect(extAnalyser);
  extAnalyser.connect(extAudioCtx.destination);

  extAudioStarted = true;
  console.log("Audio-System mit Lautstärkeregler ready!");
}

// handles volume and frame speed based on text alpha values
function updateLiveAudio() {
  requestAnimationFrame(updateLiveAudio);

  if (extAudioStarted && typeof texts !== 'undefined') {

    if (texts.length > 0) {

      // find highest alpha value currently active
      let maxAlpha = 0;
      for (let t of texts) {
        if (t.alpha > maxAlpha) {
          maxAlpha = t.alpha;
        }
      }

      // link volume to the brightest text
      let currentVolume = map(maxAlpha, 3, 255, 0.0, 1.0);
      currentVolume = constrain(currentVolume, 0, 1);

      extGainNode.gain.value = currentVolume;

      if (extAudioElement.paused) {
        extAudioCtx.resume();
        extAudioElement.play().catch(e => console.log("Start blockiert",
e));
      }
```

```
      // grab audio frequency data
      extAnalyser.getByteFrequencyData(extDataArray);
      let total = 0;
      for (let i = 0; i < extDataArray.length; i++) {
        total += extDataArray[i];
      }
      let average = total / extDataArray.length;

      // speed up animation frameCount on louder sounds
      if (average > 5) {
        let boost = map(average, 0, 255, 1, 35);
        frameCount += floor(boost);
      }

    } else {
      // pause audio if no texts are left on screen
      if (!extAudioElement.paused) {
        extGainNode.gain.value = 0;
        extAudioElement.pause();
      }
    }
  }
}
```